SystemFromScratch

OPEN-SOURCE EMBEDDED LIBRARY

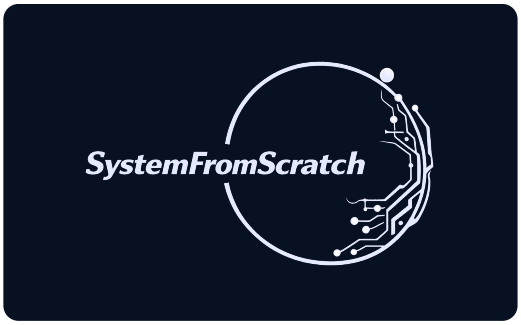
Stepper-C

Stepper Motor Control Library for Embedded Systems

USER MANUAL

Version 0.1.0 · October 2026
Anton Vynohradov
systemfromscratch.com

Imprint



Stepper-C User Manual
Library version 0.1.0
Edition: October 2026

Author
Anton Vynohradov
SystemFromScratch® — systemfromscratch.com
GitHub: github.com/AntonVynohradov
LinkedIn: linkedin.com/in/vynohradov
ORCID: orcid.org/0009-0005-6965-0911
E-mail: avynohradov@systemfromscratch.com

Online
Documentation: https://antonvynohradov.github.io/Stepper-C/
Source code: https://github.com/AntonVynohradov/Stepper-C

Copyright © 2026 Anton Vynohradov.

The Stepper-C library and this manual are licensed under the Apache License, Version 2.0 (the “License”); you may not use these files except in compliance with the License. You may obtain a copy of the License at http://www.apache.org/licenses/LICENSE-2.0 and in Appendix B. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

SystemFromScratch® and the SystemFromScratch® logo are registered trademarks. The Apache License 2.0 grants no rights to use them; they may not be used to endorse or promote products derived from Stepper-C without prior written permission.

TODO [vendor marks] are trademarks of [vendor]. They are used only to identify the protocol that Stepper-C implements. Stepper-C is an independent implementation and is not affiliated with or endorsed by [vendor]. All other trademarks are the property of their respective owners.

This manual is generated from the same sources as the online documentation. The information in it was checked against the library version stated above; later versions may differ.

Contents

1 Introduction 6

1.1 What the library does 6
1.2 What the library does not do 6
1.3 Who it is for 6
1.4 Key properties 6
1.5 Terminology 7
1.6 Conventions in this documentation 7

2 Getting Started 8

2.1 Get the library 8
2.2 Add it to your project 8
2.3 Prerequisites for development 8
2.4 Build with presets 8
2.5 Unit tests and coverage 9
2.6 Integration tests 9
2.7 Static analysis 9
2.8 Formatting 9
2.9 Versioning and releases 9
2.10 Documentation 9

3 How It Works 10

3.1 Layers 10
3.2 Data flow 10
3.3 The parser 10
3.4 The encoder 10
3.5 Memory model 11
3.6 Error handling 11
3.7 Threading and reentrancy 11
3.8 Design decisions 11

4 Using the Library 12

4.1 Setting up a link 12
4.2 Writing a device 12
4.3 Writing a controller 12
4.4 Using the codec without a link 12
4.5 Return values 12
4.6 Diagnostics 12
4.7 Common mistakes 12

5 Extending the Library 13

5.1 Where to extend 13
5.2 Adding a transport 13
5.3 Building device models 13
5.4 Building multi-device commands 13
5.5 Vendor-specific values 13
5.6 Changing the configuration 13
5.7 Contributing to the library 13

6 Configuration 14
6.1 Memory footprint 14
6.2 Stack usage 14
6.3 CMake options 14

7 Stepper Motor Control 16
7.1 The bus 16
7.2 Frame 16
7.3 Checksum 16
7.4 Escaping 16
7.5 Commands 16
7.6 Error codes 16
7.7 Parser behavior 16
7.8 Reference frames (from the specification, used in the tests) 16

8 Integration Guide 17
8.1 RTOS: interrupt, queue and RX thread 17
8.2 Adding the library to a CMake build 17
8.3 Bare metal and host 17
8.4 Notes from hardware bring-up 17

9 Examples 18
9.1 Bare metal 18
9.2 Host serial port 18
9.3 Zephyr 18

10 API Reference 19
10.1 Version — sstep.h 19
10.2 Configuration — sstep_config.h 19
10.3 Definitions — sstep_defs.h 19
10.4 Codec — sstep_protocol.h 19
10.5 Link — sstep_link.h 19
10.6 Return codes summary 19

11 MISRA C:2012 compliance 20
11.1 Checking 20
11.2 Current status 20
11.3 Manual review notes 20
11.4 Deviation register 21

Author & Company 22
Author 22
SystemFromScratch® 23
Contact 23
Support and feedback 24
Trademarks and acknowledgements 24

Appendix A Changelog 25
A.1 [Unreleased] 25

Appendix B License 27
B.1 What the license allows 27
B.2 What it requires 27

B.3 What it does not give 27
B.4 Trademarks 28
B.5 Full license text 28

CHAPTER 1

Introduction

TODO one paragraph: Stepper-C is a C11 library that implements the wire format of Stepper Motor Control for embedded systems that must be predictable and reviewable.

1.1 What the library does

- **Encodes frames.** TODO
- **Parses a byte stream.** TODO (any chunking, validation, resynchronization)
- **Connects the codec to a transport.** TODO (link, handler, write function)
- **Defines the protocol constants.** TODO

1.2 What the library does not do

Concern	Why it is not in the library
Drivers (UART, CAN, USB…)	Hardware- and OS-specific; you pass bytes in and provide a write function.
Threads, queues, interrupts, locking	Every system has its own; see the integration guide.
Device models / application data	TODO
Timeouts and retries	The library has no clock.

1.3 Who it is for

- **Device firmware.** TODO
- **Controller firmware.** TODO
- **Host tools.** TODO

1.4 Key properties

Property	Value
Language	C11, no warnings with `-Wall -Wextra -Wpedantic -Wconversion`
Dependencies	`<stddef.h>`, `<stdint.h>`, `<stdbool.h>`, `<string.h>` only
Memory	No heap; caller-owned structures; sizes fixed by `SSTEP_MAX_…`
RAM per instance	TODO measured
Stack	TODO measured (details)
Coding standard	MISRA C:2012, 0 Cppcheck addon violations, TODO deviations
Tests	100 % line and branch coverage; C/Python cross-check; specification reference frames

Property	Value
Thread safety	None built in, by design; see Threading and reentrancy
License	Apache License 2.0

1.5 Terminology

TODO term

TODO definition (controller, device, frame types, broadcast, escaping…).

1.6 Conventions in this documentation

- Hexadecimal bytes without prefix inside frames (`AA 55`), with `0x` in text and code.
- Multi-byte fields are TODO endian, as on the wire.
- `SSTEP_MAX_…` is shown with its default value.

CHAPTER 2

Getting Started

This chapter shows how to get Stepper-C, add it to your project, and build and test it.

2.1 Get the library

```
git clone https://github.com/AntonVynohradov/Stepper-C.git
```

Each release (github.com/AntonVynohradov/Stepper-C/releases) provides the source code, a prebuilt static library for Linux x86-64 with its headers, the MISRA report and the PDF user manual.

2.2 Add it to your project

CMake.

```
include(path/to/Stepper-C/sstep.cmake)
target_link_libraries(app PRIVATE sstep)
```

Any other build system. Compile `src/*.c` and add the repository root and `inc/` to the include path.

Prebuilt package. Link `lib/libsstep.a` and add `include/` to the include path.

2.3 Prerequisites for development

- C11 compiler, CMake 3.20+
- Optional: GoogleTest and gcovr, Python 3 and pytest, Cppcheck, clang-format, MkDocs (requirements-dev.txt), Doxygen, WeasyPrint with Pango

2.4 Build with presets

Preset	Purpose
`[Debug]Stepper Motor Control`	Static library with debug information
`[Release]Stepper Motor Control`	Optimized static library
`[UnitTests]Stepper Motor Control`	GoogleTest executable with coverage; target `coverage`
`[IntegrationTests]Stepper Motor Control`	pytest suite; target `integration_tests`
`[Examples]Stepper Motor Control`	Host examples and the bare-metal simulation test

2.5 Unit tests and coverage

The `coverage` target fails below `SSTEP_COVERAGE_MIN` (100 %) line or branch coverage.

2.6 Integration tests

TODO the Python twin and the C/Python cross-check.

2.7 Static analysis

Target	Tool
`cppcheck-analyze`	Cppcheck, all checks
`misra-analyze`	Cppcheck MISRA C:2012 addon; fails on any violation
`codeql-analyze`	CodeQL `cpp-queries`

2.8 Formatting

```
clang-format -i src/*.c inc/*.h sstep.h tests/unit/*.cpp examples/*/*.[ch]
```

2.9 Versioning and releases

TODO (see the skill's repository.md → Release).

2.10 Documentation

Task	Command	VS Code task
Live preview	`mkdocs serve`	*Docs: Serve on localhost*
Build the website	`mkdocs build`	*Docs: Build site*
Build the PDF manual	`python tools/docs/build_manual.py`	*Docs: Build PDF manual*
Regenerate diagrams	`python tools/docs/diagrams.py`	*Docs: Generate diagrams*

CHAPTER 3

How It Works

TODO one paragraph: what this chapter explains.

3.1 Layers

Stepper-C architecture

Layer	Header	Responsibility
Definitions	`sstep_defs.h`	TODO
Codec	`sstep_protocol.h`	TODO
Link	`sstep_link.h`	TODO

3.2 Data flow

Receive and reply data flow

Receive path. TODO

Transmit path. TODO

3.3 The parser

Parser algorithm

1. **Append.** TODO
2. **Synchronize.** TODO
3. **Check the length.** TODO
4. **Wait.** TODO
5. **Validate and decode.** TODO
6. **Reset.** TODO

> **Recovery after errors**
>
> TODO what is lost after a corrupted length, and what the controller should do.

3.4 The encoder

TODO escaping, capacity checks, checksum.

3.5 Memory model

Structure	Contents	Size with the default configuration
TODO	TODO	TODO

3.6 Error handling

Situation	Result
A `NULL` pointer or inconsistent length	`SSTEP_PARSE_ERROR`; nothing is written
TODO	TODO

3.7 Threading and reentrancy

- **Codec functions** are reentrant.
- **A link** is not thread-safe; call receive from one context, never from an ISR.
- **Sending** from several contexts needs the application's mutex.
- **The handler** data is valid only during the call.

3.8 Design decisions

Decision	Reason
One byte per parser call	Works with any transport chunking.
TODO	TODO

CHAPTER 4

Using the Library

TODO one paragraph: device, controller and codec-only roles.

4.1 Setting up a link

4.2 Writing a device

Reading parameters

Answering

Rules a well-behaved device follows

4.3 Writing a controller

4.4 Using the codec without a link

4.5 Return values

Function	Success	Failure
TODO	TODO	TODO

4.6 Diagnostics

4.7 Common mistakes

Symptom	Likely cause
TODO	TODO

CHAPTER 5

Extending the Library

TODO one paragraph: extension points; almost everything is done on top of the library.

5.1 Where to extend

You want to…	Do it in	Library change needed?
Support another transport	your write function and receive loop	No
TODO	TODO	TODO

5.2 Adding a transport

5.3 Building device models

5.4 Building multi-device commands

5.5 Vendor-specific values

5.6 Changing the configuration

5.7 Contributing to the library

1. **Code.** C11, no new dependencies, no heap, MISRA rules used in the code.
2. **Tests.** 100 % line and branch coverage; update the Python twin when the codec changes.
3. **Analysis.** `misra-analyze` 0 violations or a recorded deviation.
4. **Format.** `clang-format` reports no changes.
5. **Documentation.** Pages, stack table, diagrams, `CHANGELOG.md [Unreleased]`.
6. **Compatibility.** Public API stable within a major version.

CHAPTER 6

Configuration

Build-time parameters live in `inc/sstep_config.h`. Each one is guarded by `#ifndef` and range-checked at compile time; override it with a compiler definition.

Parameter	Default	Meaning
`SSTEP_MAX_…`	TODO	TODO (range checked at compile time)

6.1 Memory footprint

Object	Size
TODO	TODO

6.2 Stack usage

Measured with `-fstack-usage`, default configuration, bytes per function frame:

Function	Cortex-M4 `-Os`	Cortex-M4 `-O0`	x86-64 `-O2`
TODO	TODO	TODO	TODO

Toolchains: TODO versions.

Path	Chain	Stack
Transmit	TODO	TODO B + write callback
Receive	TODO	TODO B + handler

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os -std=c11 -fstack-usage -Iinc -I. -c src/*.c
cat *.su
```

6.3 CMake options

Option	Default	Meaning
`SSTEP_UNIT_TESTS`	`OFF`	GoogleTest suite and `coverage` target
`SSTEP_INTEGRATION_TESTS`	`OFF`	pytest suite
`SSTEP_EXAMPLES`	`OFF`	Host examples and the bare-metal simulation test
`SSTEP_COVERAGE_MIN`	`100`	Coverage gate
`SSTEP_MISRA_FAIL_ON_VIOLATION`	`ON`	MISRA gate

Option	Default	Meaning
`SSTEP_MISRA_RULE_TEXTS`	empty	Licensed MISRA rule texts

CHAPTER 7

Stepper Motor Control

This chapter describes the Stepper Motor Control wire format and exactly what Stepper-C does with each part of it. The normative reference is the specification.

7.1 The bus

Bus topology

7.2 Frame

Frame structure

Field	Offset	Size	Meaning
TODO	TODO	TODO	TODO

7.3 Checksum

7.4 Escaping

7.5 Commands

7.6 Error codes

7.7 Parser behavior

7.8 Reference frames (from the specification, used in the tests)

Frame	Bytes
TODO	TODO

CHAPTER 8

Integration Guide

The library does not touch hardware. An integration has three parts:

1. **Receive:** move bytes from the transport into `sstep_link_receive()` from a thread or the main loop, not from an interrupt handler.
2. **Transmit:** a write function that sends a complete frame; serialize it if several threads send.
3. **Application:** a handler that interprets frames and answers.

Complete, buildable programs for each pattern are described in Examples (p. 18).

8.1 RTOS: interrupt, queue and RX thread

8.2 Adding the library to a CMake build

```
include(path/to/Stepper-C/sstep.cmake)
target_link_libraries(app PRIVATE sstep)
target_link_libraries(sstep PUBLIC zephyr_interface)   # Zephyr applications only
```

8.3 Bare metal and host

8.4 Notes from hardware bring-up

CHAPTER 9

Examples

Example	Role	Platform	Built and run in CI
Bare metal	Device	Any MCU; runs on a PC against a simulated board	Yes, as a self-checking test
Host serial port	Control-ler	Linux, macOS, Windows	Built
Zephyr	Device	Any Zephyr board	No, needs a Zephyr work-space

```
cmake --preset "[Examples]Stepper Motor Control"
cmake --build "build/[Examples]Stepper Motor Control"
ctest --test-dir "build/[Examples]Stepper Motor Control" -L example --output-on-failure
```

9.1 Bare metal

9.2 Host serial port

9.3 Zephyr

CHAPTER 10

API Reference

This chapter documents every public function, type and macro of Stepper-C. Include the umbrella header:

```
#include "sstep.h"
```

Header	Contents
`sstep.h`	All headers; library version
`sstep_config.h`	Build-time configuration
`sstep_defs.h`	Protocol constants, byte-order helpers
`sstep_protocol.h`	Checksum, stream parser, encoder
`sstep_link.h`	Transport-independent link

10.1 Version — `sstep.h`

10.2 Configuration — `sstep_config.h`

10.3 Definitions — `sstep_defs.h`

10.4 Codec — `sstep_protocol.h`

TODO for every function: prototype in a ```` ```c ```` block, one paragraph, a Parameter | Description table and a Return | Meaning table.

10.5 Link — `sstep_link.h`

10.6 Return codes summary

Value	Meaning
`0`	Success
`SSTEP_PARSE_ERROR` (`-1`)	Invalid argument, malformed frame or insufficient capacity

CHAPTER 11

MISRA C:2012 compliance

This page records how the library is checked against MISRA C:2012 and every deviation from it. It covers the library code only: `src/`, `inc/` and `sstep.h`.

11.1 Checking

Check	Where	Gate
Cppcheck MISRA C:2012 addon	`misra-analyze`, CI *MISRA C:2012 Analysis*	Fails on any violation
Cppcheck, all checks	`cppcheck-analyze`, CI *Cppcheck Analysis*	Report
CodeQL `cpp-queries`	`codeql-analyze`, CI *CodeQL Analysis*	Report
Compiler	`-std=c11 -Wall -Wextra -Wpedantic -Wconversion`	No warnings
Manual review	Pull request review	Reviewer

11.2 Current status

Item	Result
Cppcheck MISRA addon	TODO violations (versions)
Unit test coverage of `src/`	TODO
Deviations	TODO

11.3 Manual review notes

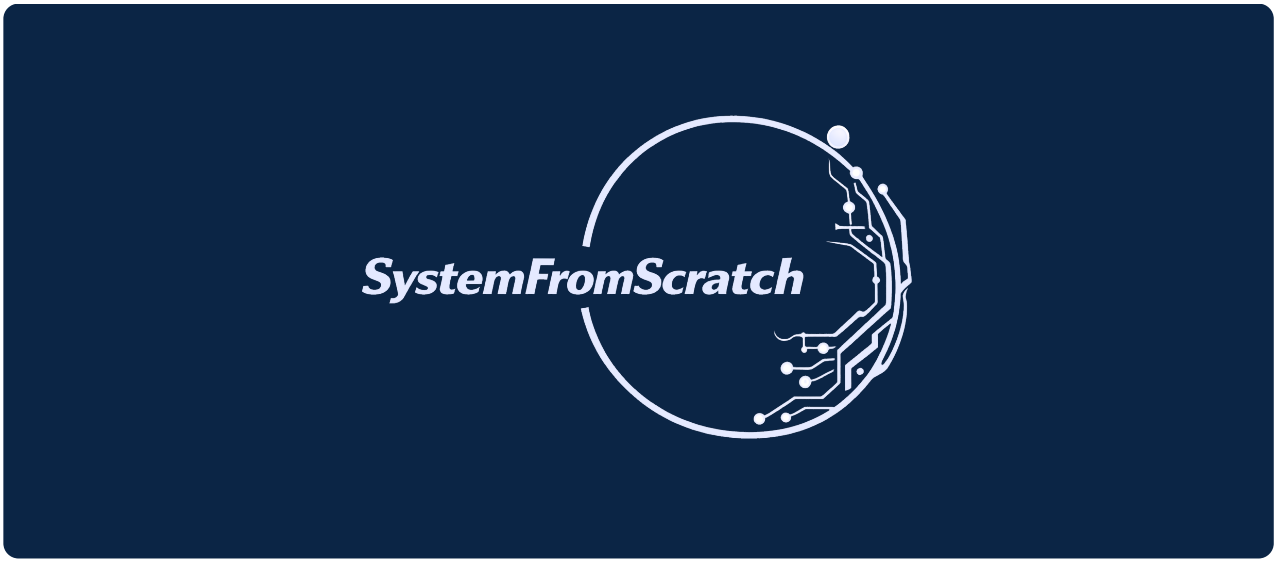
Guideline	How the library complies
Dir 4.1 (run-time failures)	TODO
Dir 4.11 (validity of arguments)	TODO
Dir 4.12, Rule 21.3 (dynamic memory)	No heap.
Rule 15.5 (single point of exit)	One `return` per function.
Rule 17.2 (recursion)	No recursion.
Rule 21.6 (standard I/O)	No `<stdio.h>`.

11.4 Deviation register

DEV-01: TODO (Rule/Dir, category)

- **Location:** TODO
- **Rationale:** TODO
- **Risk control:** TODO

Author & Company



Stepper-C is designed, written and maintained by **Anton Vynohradov** and published by **SystemFromScratch®** — secure embedded systems and compliance-ready engineering, from concept to certification.

Author

Anton Vynohradov · ORCID 0009-0005-6965-0911 (orcid.org/0009-0005-6965-0911)

Embedded Systems Engineer · Secure Product Architect · Founder of SystemFromScratch®

Embedded systems built from scratch — from bare metal to Linux, from first schematic to certification.

Anton designs and builds reliable, secure embedded products end to end: firmware, board support packages and drivers, RTOS and Embedded Linux platforms, hardware bring-up, and the security and compliance work needed to ship them. He is based in Poland and works remotely worldwide.

Area	Focus
Embedded software	Bare-metal, RTOS and Linux firmware in C/C++, built to be maintained for years
BSP & drivers	Bootloaders, Zephyr drivers and Linux kernel work for MCUs, SoCs and SoC FPGAs
Security & compliance	Secure boot, threat modelling, MISRA C/C++, EU Cyber Resilience Act (CRA) readiness
Industries	Industrial IoT, automotive, robotics, medical and consumer electronics

Technology: C · C++ · Python · Zephyr RTOS · FreeRTOS · Embedded Linux · Yocto · Buildroot · ARM Cortex-M · STM32 · Microchip FPGA · RISC-V · CAN / ISO-TP · ROS 2 · Docker · Git · CI/CD · MISRA C/C++ · EU CRA

For Stepper-C, Anton wrote the library, its unit and integration tests, the examples, the desktop tool (if any) and this documentation.

SystemFromScratch®

SystemFromScratch® was founded by Anton Vynohradov. The company combines deep embedded engineering with cybersecurity and certification-focused product development, for teams in industrial IoT, automotive, robotics, medical and consumer electronics that need a partner who understands the whole stack — silicon, firmware, operating system and the regulations that govern them.

Service	What it covers
Embedded software development	Bare-metal, RTOS and Linux firmware for performance, reliability and long-term maintainability
BSP & device drivers	Board support packages, bootloaders and low-level drivers for MCUs, SoCs, FPGAs and custom hardware
RTOS & Embedded Linux	Zephyr, FreeRTOS and Embedded Linux (Yocto, Buildroot): porting, configuration, integration, reproducible builds
Hardware bring-up & integration	Schematic review, first boot, validation and production support, including SoC FPGA and motor-control platforms
Cybersecurity & CRA readiness	Secure boot, firmware integrity, threat modelling and a secure development lifecycle for the EU Cyber Resilience Act
Compliance & consulting	MISRA C/C++, static analysis, code and architecture reviews, compliance roadmaps

Stepper-C is one of the open-source projects in the SystemFromScratch® portfolio: public engineering work that can be inspected line by line, built with the same rules — MISRA C, full test coverage, documented decisions — that the company applies to customer products.

Contact

Channel	Address
Website	systemfromscratch.com
E-mail	avynohradov@systemfromscratch.com
GitHub	github.com/AntonVynohradov
GitHub organization	github.com/SystemFromScratch
LinkedIn	linkedin.com/in/vynohradov
ORCID	0009-0005-6965-0911 (orcid.org/0009-0005-6965-0911)
Location	Poland — working remotely worldwide

Building a connected product, bringing up new hardware, or preparing for the Cyber Resilience Act? Get in touch through systemfromscratch.com or by e-mail.

Support and feedback

- **Bugs and feature requests:** open an issue on GitHub (github.com/AntonVynohradov/Stepper-C/issues). Please include the library version, your compiler and target, and — for communication problems — a capture of the bytes on the bus.
- **Contributions:** see Contributing to the library (p. 13).
- **Commercial support, integration and custom development:** contact SystemFromScratch® through systemfromscratch.com.

Trademarks and acknowledgements

SystemFromScratch® is a registered trademark. The SystemFromScratch® name and logo are used in this documentation to identify the publisher of Stepper-C; the Apache License 2.0 grants no rights to use them, and they may not be used to endorse or promote products derived from Stepper-C without prior written permission.

Stepper-C implements the protocol as published by SystemFromScratch in the Stepper Motor Control specification. The reference packets used in the tests come from that specification. TODO [vendor marks] are trademarks of SystemFromScratch; Stepper-C is an independent implementation and is not affiliated with or endorsed by SystemFromScratch. (Delete this paragraph for an open standard without trademarks.)

APPENDIX A

Changelog

All notable changes to this project are documented in this file. The format is based on Keep a Changelog (keepachangelog.com/en/1.1.0/), and this project adheres to Semantic Versioning (semver.org/spec/v2.0.0.html).

A.1 [Unreleased]

Added

- Repository skeleton from the SystemFromScratch® library blueprint (CMake, presets, CI workflows, VS Code tasks, documentation site and PDF manual tooling).
- Public API contract (headers only, no implementation yet):
- `inc/sstep_config.h`: build-time configuration (numeric backend, topologies, microstep limit, sine table size, argument checks), each value range-checked;
- `inc/sstep_defs.h`: status codes, topologies, fault flags, driver states;
- `inc/sstep_math.h`: backend types (Q15 `int16_t` or `float`), per-unit convention, 32-bit electrical angle;
- `inc/sstep_ctrl.h`: configuration structures in integer engineering units, hardware callbacks given at initialisation, fast/slow loop functions, motion commands.
- CMake cache variables `SSTEP_MATH_BACKEND` (FIXED/FLOAT) and `SSTEP_CONFIG_DIR`.
- `tests/compile/sstep_api_check.c`: compile check of the API with an example configuration (17HS19-2004S1 on B-G431B-ESC1).
- Numeric backend (`inc/sstep_math.h`, `src/sstep_math.c`):
- saturating `static inline` operations for both backends (add, sub, neg, abs, mul, clamp, accumulator add/clamp, gain multiply), identical ranges in both backends;
- `struct sstep_gain` (Q15 mantissa + exponent, or float) and `sstep_gain_from_ratio()`, `sstep_real_from_ratio()`, `sstep_real_to_units()` for conversions from engineering units without floating point;
- `sstep_sin_cos()`: quarter-wave table with linear interpolation; the table (`src/sstep_sin_table.c`) is generated by `tools/sstep_gen_sin_table.py`.

- Preset `[UnitTestsFloat]Stepper Motor Control`; `[UnitTests]Stepper Motor Control` builds the fixed point backend.

- Control implementation (both numeric backends):

- `sstep_init()`: validation of every configuration field, per-unit gains from engineering units without floating point (Kp = L wc, Ki = R wc, back-calculation Kb = Ts R / L, ADC scaling, back-EMF feed-forward; 2 pi = 710 / 113);
- fast loop: ADC conversion, overcurrent / overvoltage / undervoltage trips, PI current control with Q27 integrators (fixed point) and back-calculation anti-windup, voltage drive mode, averaged power stage for two H-bridges and for three legs with a common leg (voltage vector scaled with its direction kept, bus voltage compensation);

- slow loop: integer motion generator (positions in 1/65536 microstep, exact discrete braking: no overshoot, exact stop on the target), hold current, events, bus voltage scale; angle interpolated in the fast loop;
- slow -> fast hand-off through a double buffer (no torn reads on a single core);
- commands: move to / by, run, stop, set position (rotor angle kept), currents, motion limits, faults, status.
- `sstep_gain_from_ratio64()` (at most three 64-bit divisions), integrator type `sstep_integ_t`, `sstep_gain_mul_acc()`, `sstep_div_unit()`, `sstep_acc_scale()`, `sstep_acc_to_ticks()`.
- No dependency on `<string.h>`.

Quality

- Unit tests `tests/unit/gtest_sstep_math.cpp` (18 tests): 100 % line and branch coverage of `src/sstep_math.c` and of the inline operations, in both backends.
- Measured sin/cos error, default table (8 bits): fixed point 1.0 LSB (3.05e-5), float 4.7e-6.
- Unit tests of the power stage, motion generator, API and closed loop with a motor model (`tests/unit/sstep_test_plant.hpp`): 76 tests (fixed point), 75 (float); 100 % line and branch coverage of `src/` in both backends (gcov); clean under ASan and UBSan.
- Closed loop against the Scilab prototype (sfs-stepper-sim): current step overshoot 1.11 %, settling 0.40 ms (prototype 1.12 %, 0.40 ms); 8 rev/s current 1.862 A two H-bridges, 1.557 A shared leg (prototype 1.862 / 1.557 A); load angle at 0.25 N*m 37.21 deg (prototype 37.21 deg).
- Code size (clang -Os, both topologies, sine table included): Cortex-M0+ fixed point 8.5 kB, Cortex-M4F fixed point 8.2 kB, Cortex-M4F float 9.0 kB.

APPENDIX B

License

Stepper-C is free and open-source software, licensed under the **Apache License, Version 2.0 (apache.org/licenses/LICENSE-2.0)**.

Item	Value
Copyright	© 2026 Anton Vynohradov
License	Apache License 2.0
SPDX identifier	`Apache-2.0`
Applies to	The library (`sstep.h`, `inc/`, `src/`), tests, examples, tools and this documentation

B.1 What the license allows

In short — the full text below is authoritative:

- **Use** the library in any project, open source or proprietary, commercial or not.
- **Modify** it and **distribute** it, in source or binary form, alone or as part of a product.
- **Patent grant:** each contributor grants you a license to any of their patents that the contribution uses.

B.2 What it requires

When you distribute Stepper-C or a modified version of it:

- Include a copy of the license.
- Keep the copyright, patent, trademark and attribution notices in the source files.
- Mark the files you changed.
- If you redistribute a NOTICE file, keep its attribution notices (Stepper-C has none at the moment).

Firmware that links the library in binary form only needs to make the license and the copyright notice available, for example in its documentation or "about" screen.

B.3 What it does not give

- No warranty: the software is provided "as is".
- No trademark rights: the license does not let you use the names of the author or of SystemFromScratch® to endorse your product.

B.4 Trademarks

SystemFromScratch® is a registered trademark; the license grants no rights to use the name or logo. TODO [vendor marks] are trademarks of SystemFromScratch. They are used in this documentation only to identify the protocol that Stepper-C implements. Stepper-C is an independent implementation and is not affiliated with or endorsed by SystemFromScratch.

B.5 Full license text


```
                                 Apache License
                           Version 2.0, January 2004
                        http://www.apache.org/licenses/

   TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

   1. Definitions.

      "License" shall mean the terms and conditions for use, reproduction,
      and distribution as defined by Sections 1 through 9 of this document.

      "Licensor" shall mean the copyright owner or entity authorized by
      the copyright owner that is granting the License.

      "Legal Entity" shall mean the union of the acting entity and all
      other entities that control, are controlled by, or are under common
      control with that entity. For the purposes of this definition,
      "control" means (i) the power, direct or indirect, to cause the
      direction or management of such entity, whether by contract or
      otherwise, or (ii) ownership of fifty percent (50%) or more of the
      outstanding shares, or (iii) beneficial ownership of such entity.

      "You" (or "Your") shall mean an individual or Legal Entity
      exercising permissions granted by this License.

      "Source" form shall mean the preferred form for making modifications,
      including but not limited to software source code, documentation
      source, and configuration files.

      "Object" form shall mean any form resulting from mechanical
      transformation or translation of a Source form, including but
      not limited to compiled object code, generated documentation,
      and conversions to other media types.

      "Work" shall mean the work of authorship, whether in Source or
      Object form, made available under the License, as indicated by a
      copyright notice that is included in or attached to the work
      (an example is provided in the Appendix below).

      "Derivative Works" shall mean any work, whether in Source or Object
      form, that is based on (or derived from) the Work and for which the
      editorial revisions, annotations, elaborations, or other modifications
      represent, as a whole, an original work of authorship. For the purposes
      of this License, Derivative Works shall not include works that remain
      separable from, or merely link (or bind by name) to the interfaces of,
      the Work and Derivative Works thereof.

      "Contribution" shall mean any work of authorship, including
      the original version of the Work and any modifications or additions
      to that Work or Derivative Works thereof, that is intentionally
      submitted to Licensor for inclusion in the Work by the copyright owner
      or by an individual or Legal Entity authorized to submit on behalf of
      the copyright owner. For the purposes of this definition, "submitted"
      means any form of electronic, verbal, or written communication sent
      to the Licensor or its representatives, including but not limited to
      communication on electronic mailing lists, source code control systems,
      and issue tracking systems that are managed by, or on behalf of, the
      Licensor for the purpose of discussing and improving the Work, but
      excluding communication that is conspicuously marked or otherwise
      designated in writing by the copyright owner as "Not a Contribution."
```

"Contributor" shall mean Licensor and any individual or Legal Entity
on behalf of whom a Contribution has been received by Licensor and
subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of
this License, each Contributor hereby grants to You a perpetual,
worldwide, non-exclusive, no-charge, royalty-free, irrevocable
copyright license to reproduce, prepare Derivative Works of,
publicly display, publicly perform, sublicense, and distribute the
Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of
this License, each Contributor hereby grants to You a perpetual,
worldwide, non-exclusive, no-charge, royalty-free, irrevocable
(except as stated in this section) patent license to make, have made,
use, offer to sell, sell, import, and otherwise transfer the Work,
where such license applies only to those patent claims licensable
by such Contributor that are necessarily infringed by their
Contribution(s) alone or by combination of their Contribution(s)
with the Work to which such Contribution(s) was submitted. If You
institute patent litigation against any entity (including a
cross-claim or counterclaim in a lawsuit) alleging that the Work
or a Contribution incorporated within the Work constitutes direct
or contributory patent infringement, then any patent licenses
granted to You under this License for that Work shall terminate
as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the
Work or Derivative Works thereof in any medium, with or without
modifications, and in Source or Object form, provided that You
meet the following conditions:

   (a) You must give any other recipients of the Work or
       Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent notices
       stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Works
       that You distribute, all copyright, patent, trademark, and
       attribution notices from the Source form of the Work,
       excluding those notices that do not pertain to any part of
       the Derivative Works; and

   (d) If the Work includes a "NOTICE" text file as part of its
       distribution, then any Derivative Works that You distribute must
       include a readable copy of the attribution notices contained
       within such NOTICE file, excluding those notices that do not
       pertain to any part of the Derivative Works, in at least one
       of the following places: within a NOTICE text file distributed
       as part of the Derivative Works; within the Source form or
       documentation, if provided along with the Derivative Works; or,
       within a display generated by the Derivative Works, if and
       wherever such third-party notices normally appear. The contents
       of the NOTICE file are for informational purposes only and
       do not modify the License. You may add Your own attribution
       notices within Derivative Works that You distribute, alongside
       or as an addendum to the NOTICE text from the Work, provided
       that such additional attribution notices cannot be construed
       as modifying the License.

   You may add Your own copyright statement to Your modifications and
   may provide additional or different license terms and conditions
   for use, reproduction, or distribution of Your modifications, or
   for any such Derivative Works as a whole, provided Your use,
   reproduction, and distribution of the Work otherwise complies with
   the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
any Contribution intentionally submitted for inclusion in the Work
by You to the Licensor shall be under the terms and conditions of
this License, without any additional terms or conditions.
Notwithstanding the above, nothing herein shall supersede or modify
the terms of any separate license agreement you may have executed
with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade

```
      names, trademarks, service marks, or product names of the Licensor,
      except as required for reasonable and customary use in describing the
      origin of the Work and reproducing the content of the NOTICE file.

   7. Disclaimer of Warranty. Unless required by applicable law or
      agreed to in writing, Licensor provides the Work (and each
      Contributor provides its Contributions) on an "AS IS" BASIS,
      WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
      implied, including, without limitation, any warranties or conditions
      of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
      PARTICULAR PURPOSE. You are solely responsible for determining the
      appropriateness of using or redistributing the Work and assume any
      risks associated with Your exercise of permissions under this License.

   8. Limitation of Liability. In no event and under no legal theory,
      whether in tort (including negligence), contract, or otherwise,
      unless required by applicable law (such as deliberate and grossly
      negligent acts) or agreed to in writing, shall any Contributor be
      liable to You for damages, including any direct, indirect, special,
      incidental, or consequential damages of any character arising as a
      result of this License or out of the use or inability to use the
      Work (including but not limited to damages for loss of goodwill,
      work stoppage, computer failure or malfunction, or any and all
      other commercial damages or losses), even if such Contributor
      has been advised of the possibility of such damages.

   9. Accepting Warranty or Additional Liability. While redistributing
      the Work or Derivative Works thereof, You may choose to offer,
      and charge a fee for, acceptance of support, warranty, indemnity,
      or other liability obligations and/or rights consistent with this
      License. However, in accepting such obligations, You may act only
      on Your own behalf and on Your sole responsibility, not on behalf
      of any other Contributor, and only if You agree to indemnify,
      defend, and hold each Contributor harmless for any liability
      incurred by, or claims asserted against, such Contributor by reason
      of your accepting any such warranty or additional liability.

   END OF TERMS AND CONDITIONS

   APPENDIX: How to apply the Apache License to your work.

      To apply the Apache License to your work, attach the following
      boilerplate notice, with the fields enclosed by brackets "[]"
      replaced with your own identifying information. (Don't include
      the brackets!)  The text should be enclosed in the appropriate
      comment syntax for the file format. We also recommend that a
      file or class name and description of purpose be included on the
      same "printed page" as the copyright notice for easier
      identification within third-party archives.

   Copyright [yyyy] [name of copyright owner]

   Licensed under the Apache License, Version 2.0 (the "License");
   you may not use this file except in compliance with the License.
   You may obtain a copy of the License at

       http://www.apache.org/licenses/LICENSE-2.0

   Unless required by applicable law or agreed to in writing, software
   distributed under the License is distributed on an "AS IS" BASIS,
   WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
   See the License for the specific language governing permissions and
   limitations under the License.
```